

Claude-Driven Platform

AI-Native Infrastructure Portfolio

devyoushin

2026-05-17

Contents

Claude-Driven Platform — AI-Native Infrastructure Portfolio	1
1. 프로젝트 개요	1
2. 아키텍처	2
3. 네트워크 설계	3
4. 보안 설계	3
5. 컴퓨팅 & 데이터	4
6. 모니터링 & 알람	5
7. CI/CD 파이프라인	5
8. 태그 전략	6
9. AI-Native 워크플로우	6
10. 프로젝트 구조	7
11. 향후 계획	7
부록: Architecture Decision Records	8

Claude-Driven Platform — AI-Native Infrastructure Portfolio

1. 프로젝트 개요

목적

AI(Claude)와의 협업을 통해 **엔터프라이즈 수준의 AWS 인프라를 처음부터 끝까지 설계·구축한** 포트폴리오 프로젝트.

핵심 가치

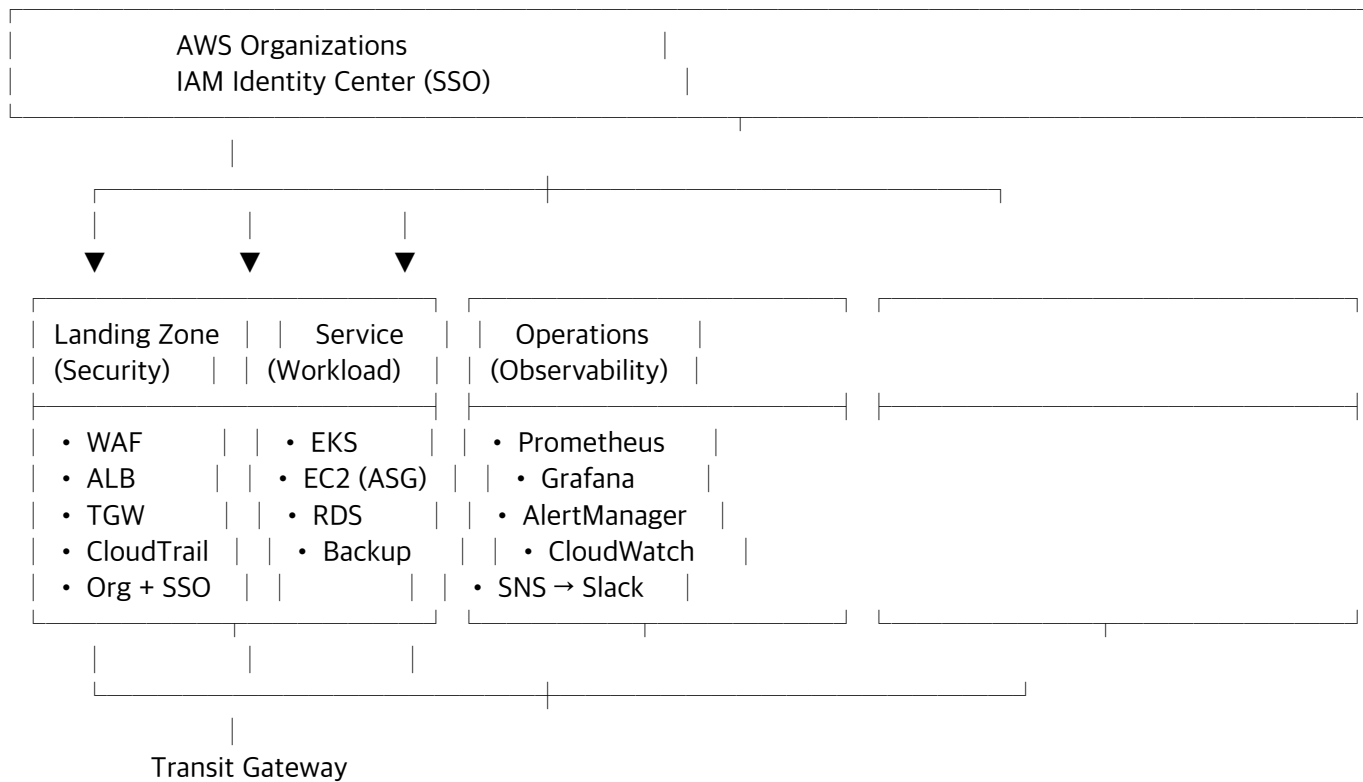
- **AI-Native 워크플로우**: 설계부터 구현까지 Claude와 페어 프로그래밍
- **Production-Ready**: 실제 운영 가능한 수준의 코드 품질
- **문서화**: 모든 의사결정 과정을 ADR로 기록

기술 스택

영역	기술
Cloud	AWS (ap-northeast-2)
IaC	Terraform >= 1.5
Container	EKS (Kubernetes 1.29)
Monitoring	Prometheus + Grafana + AlertManager
CI/CD	GitHub Actions + OIDC
Security	WAF, SCP, Identity Center, CloudTrail

2. 아키텍처

Multi-Account 전략 (3-Account)



왜 이 구조인가?

원칙	적용
관심사 분리	보안 / 서비스 / 모니터링을 계정 단위로 분리
장애 격리	서비스 장애 시에도 모니터링은 독립 동작
최소 권한	계정 간 접근은 AssumeRole (읽기 전용)
확장성	서비스 계정 추가 시 TGW attachment만 연결

3. 네트워크 설계

VPC / Subnet 구성

Account	VPC CIDR	Public	Private App	Private DB
Landing Zone	10.0.0.0/16	10.0.1-2.0/24	10.0.11-12.0/24	—
Service	10.10.0.0/16	10.10.1-2.0/24	10.10.11-12.0/24	10.10.21-22.0/24
Operations	10.20.0.0/16	10.20.1-2.0/24	10.20.11-12.0/24	—

트래픽 흐름

User → WAF → ALB (Landing Zone)
→ Transit Gateway
→ Service VPC → EKS/EC2 → RDS

Operations → TGW → Service VPC (메트릭 scrape)

- AZ 2개 (ap-northeast-2a, 2c) 사용으로 고가용성 확보
- NAT Gateway AZ별 배치 (Single AZ 장애 대비)
- DB subnet은 인터넷 라우팅 없음 (격리)

4. 보안 설계

IAM 전략: Zero Trust

사용자 → SSO Portal → Permission Set → 대상 Account
(MFA 필수) (시간 제한) (최소 권한)

설계 원칙: - IAM User 생성 금지 (SCP로 강제) - Long-lived Access Key 없음 (OIDC + 임시 Credential) - Root 사용자 사용 금지 (SCP로 차단)

Permission Sets (역할 기반)

Group	대상 Account	권한	제한
platform-admins	Landing Zone	Full Admin	—
developers	Service	EKS/EC2/S3	삭제 불가
sre-team	Operations	모니터링 Full	인프라 삭제 불가
sre-team	Service	Read Only	디버깅용
viewers	All	View Only	감사 전용

Service Control Policies

SCP	효과
Region Restriction	ap-northeast-2만 허용
Deny Root	Root 사용자 완전 차단
Deny IAM User	IAM User 생성 불가
Protect CloudTrail	감사 로그 삭제/중지 방지
Tag Policy	필수 태그 강제

감사 (Audit)

- CloudTrail: 조직 전체 API 호출 기록
 - S3 저장 (90일 후 Glacier 아카이빙, 365일 보관)
 - CloudWatch Logs 실시간 전달
-

5. 컴퓨팅 & 데이터

EKS (Kubernetes)

- Managed Node Group (t3.medium, Auto Scaling)
- Secrets encryption (KMS)
- 전체 Control Plane 로그 활성화
- IRSA (Pod 단위 IAM 권한)

EC2 (Auto Scaling Group)

- Launch Template + ASG
- IMDSv2 강제 (보안)
- SSM 접근 (SSH Key 불필요)
- CloudWatch Agent 자동 설치

RDS (PostgreSQL)

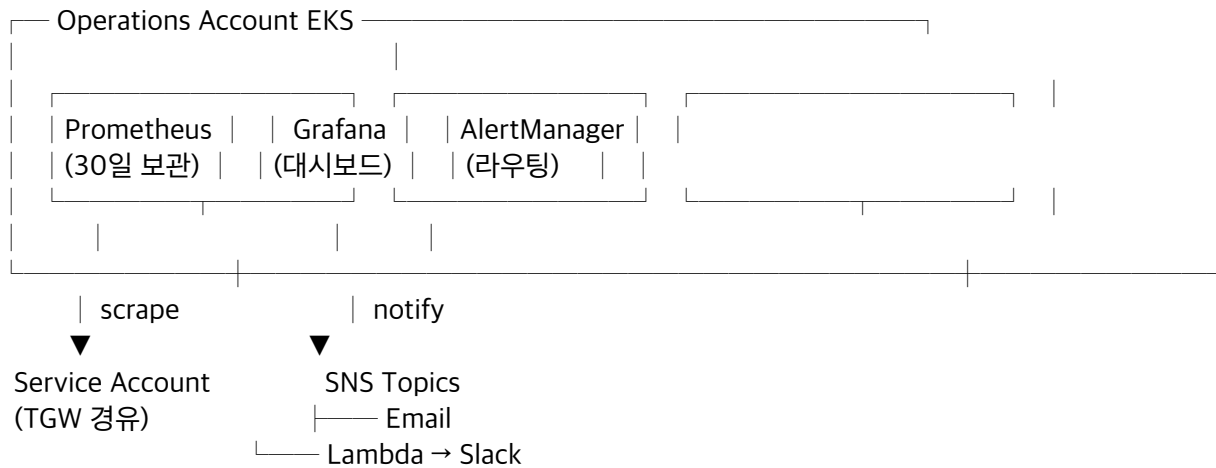
- Multi-AZ 자동 failover
- Storage encryption (KMS)
- Performance Insights 활성화
- Enhanced Monitoring (60초 간격)
- 자동 백업 7일 보관

AWS Backup

- Daily: 7일 보관
 - Weekly: 30일 보관
 - Monthly: 365일 보관 (30일 후 Cold Storage)
 - 태그 기반 자동 대상 선정 (Backup = true)
-

6. 모니터링 & 알람

스택 구성



알람 정책

Severity	대상	예시
Critical	Email + Slack 즉시	서비스 다운, RDS failover
Warning	Email	CPU 80%+, 디스크 70%+
Info	대시보드만	배포 완료, 스케일링 이벤트

CloudWatch Dashboard

- Cross-account 메트릭 통합 표시
- EKS, RDS, ALB 핵심 지표 한 화면

7. CI/CD 파이프라인

흐름

코드 변경 → PR 생성 → 자동 검증 → 리뷰 → Merge → 자동 배포

PR 단계 (자동)

검증	도구	역할
Plan	Terraform	변경 계획 확인
Security	tfsec + Checkov	IaC 보안 취약점
Format	terraform fmt	코드 스타일
Cost	Infracost	비용 영향 추정

Deploy 단계 (승인 후)

Landing Zone (승인) → Service (승인) → Operations (승인)

- 의존성 순서 준수
- GitHub Environment로 수동 승인 게이트
- 실패 시 후속 배포 자동 중단

Drift Detection

- 매일 09:00 KST 자동 실행
- terraform plan -detailed-exitcode로 수동 변경 감지
- Drift 발견 시 GitHub Issue 자동 생성

인증: OIDC (Keyless)

- AWS Access Key 완전 제거
- GitHub → OIDC Token → AWS AssumeRole
- Job 실행 시 임시 credential 발급 (자동 만료)

8. 태그 전략

모든 리소스에 일관된 태그 적용:

Tag	값	목적
Project	claude-driven-platform	프로젝트 식별
Environment	dev / staging / prod	환경 구분
ManagedBy	terraform	IaC 관리 표시
Owner	devyoushin	책임자
CostCenter	platform	비용 추적
Component	landing-zone / service / operations	계정 구분

- Tag Policy로 Organizations 수준에서 강제
- 비용 추적, 보안 감사, 자동화에 활용

9. AI-Native 워크플로우

Claude와의 협업 방식

단계	사람의 역할	Claude의 역할
설계	요구사항, 제약조건 제시	아키텍처 옵션 제안, 트레이드오프 분석
의사결정	최종 판단	ADR 문서 작성, 장단점 정리
구현	방향 확인, 코드 리뷰	Terraform 코드 작성
문서화	확인 및 보완	history.md, STATUS.md 업데이트

증거

- 모든 커밋: Co-Authored-By: Claude Opus 4.6
- ADR 문서: 의사결정 과정 기록
- history.md: 실행한 모든 명령어 기록
- Git history: AI 협업의 실시간 흔적

핵심 인사이트

“AI는 코드 생성기가 아니라 **아키텍처 파트너**로 활용할 때 가장 효과적이다.”

- CLAUDE.md로 프로젝트 컨텍스트를 지속 제공
- 단순 코드 생성이 아닌 **설계 토론 → 결정 → 구현** 흐름
- 사람은 “왜”를 결정하고, AI는 “어떻게”를 실행

10. 프로젝트 구조

```
claude-driven-platform/
├── .github/workflows/      # CI/CD 파이프라인 (4개)
├── infra/terraform/
│   ├── modules/           # 재사용 모듈 (VPC, TGW, Tags)
│   ├── landing-zone/      # Security Account (8개 파일)
│   ├── service/           # Workload Account (9개 파일)
│   └── operations/        # Observability Account (9개 파일)
├── docs/
│   ├── decisions/         # ADR 4개 + INDEX
│   ├── history.md         # 상세 작업 로그
│   ├── ai-workflow.md     # AI 협업 방법론
│   └── portfolio-presentation.md # 이 문서
├── STATUS.md              # 현재 진행 상태
├── CLAUDE.md              # AI 컨텍스트
└── README.md              # 프로젝트 소개
```

11. 향후 계획

항목	설명
AWS 실제 배포	terraform apply로 전체 인프라 프로비저닝
보안 강화	Security Hub, GuardDuty, AWS Config
샘플 앱 배포	EKS에 컨테이너 앱 + CI/CD 연동
Grafana 대시보드	운영 가시성 확보
비용 최적화	Savings Plans, 리소스 사이징

부록: Architecture Decision Records

#	제목	핵심 결정
001	Multi-Account Architecture	3계정 분리 (Landing Zone / Service / Operations)
002	Monitoring Account Separation	모니터링을 별도 계정으로 → 장애 격리
003	Centralized IAM	Organizations + Identity Center로 SSO 중앙 관리
004	CI/CD Pipeline	GitHub Actions + OIDC (Access Key 제거)

Built with Claude · AI-Native Infrastructure Portfolio GitHub: github.com/devyoushin/claude-driven-platform